PROYECCIÓN SOCIAL | ENTREGA SEMANA 5

RECICLA CERCA

EcoRedSV

Implementación de modelos, relaciones y migraciones en Django

Las clases del sistema convertidas en modelos de Django, con sus relaciones, sus tablas creadas en SQL Server y datos de prueba ingresados desde el panel de administración.

Equipo	Grupo 6
Proyecto	Recicla Cerca
Comunidad de referencia	Colonia Veracruz, Chalatenango
Asignatura	Programación Orientada a Objetos
Fecha	Octubre de 2026
Docente / institución	Francisco Daniel Peñate Mena / Universidad Dr. Andrés Bello

INTEGRANTES

Carlos Fernando Guardado García
Liliana Dalila Ulloa González
Daniela Ester López Varela
Robin Christopher Barrera Vásquez
Alexis Omar García Romero

Contenido

1. Datos de la semana y modalidad de trabajo
2. Distribución de responsabilidades
3. Estructura actual de las aplicaciones
4. Modelos y relaciones
5. Aplicación de Programación Orientada a Objetos
6. Ejecución de las migraciones
7. Panel de administración y datos de prueba
8. Historial de commits
9. Dificultades encontradas y soluciones
10. Enlaces

1. Datos de la semana y modalidad de trabajo

En la Semana 5 las clases del sistema se convirtieron en modelos de Django. Cada modelo generó su tabla en SQL Server mediante migraciones, con los nombres de tablas y columnas definidos en el diccionario de datos de la Semana 3.

Semana	Semana 5 de 8
Tema	Implementación de modelos, relaciones y migraciones en Django
Punto de partida	Diagrama de clases y diccionario de datos de la Semana 3; lógica en Python de la Semana 4

Objetivo de la semana: implementar las clases principales como modelos de Django, establecer sus relaciones y crear las tablas correspondientes en SQL Server.

Modalidad de trabajo: cada integrante desarrolla la actividad de su rol y una sola persona integra los aportes en este documento. La participación se evidencia en la tabla de responsabilidades, en los comentarios de revisión dentro del documento y en los commits en GitHub.

2. Distribución de responsabilidades

N.º	Integrante	Rol	Actividad de la Semana 5	Evidencia
1	Carlos Fernando Guardado García	Desarrollo y base de datos	Modelos y relaciones, reinicio de la base de datos, migraciones, panel de administración, datos de prueba e integración del documento.	Commits del proyecto y secciones 3 a 9

N.º	Integrante	Rol	Actividad de la Semana 5	Evidencia
2	Liliana Dalila Ulloa González	Análisis y documentación	Revisión de que los modelos y las tablas coincidan con el diccionario de datos y los requisitos de la Semana 3.	Comentarios de revisión en el documento
3	Daniela Ester López Varela	Diseño e interfaz	Revisión del panel de administración y de la página de inicio, con propuestas para la interfaz de la Semana 6.	Comentarios de revisión en el documento
4	Robin Christopher Barrera Vásquez	Pruebas y calidad	Subir el PDF de la Semana 5 a la carpeta docs del repositorio.	Commit en GitHub
5	Alexis Omar García Romero	Gestión y evidencias	Actualizar el README con el estado del proyecto y la creación del superusuario.	Commit en GitHub

Además, cada integrante deja comentarios de retroalimentación a lo largo de este documento, en las secciones relacionadas con su rol.

3. Estructura actual de las aplicaciones

Las cinco aplicaciones de la Semana 4 se mantienen, cada una con una responsabilidad del sistema. En esta semana cada aplicación recibió sus modelos (models.py), su registro en el panel (admin.py) y sus migraciones (carpeta migrations).

Aplicación	Responsabilidad	Modelos
core	Clases base y página de inicio	EntidadBase y Ubicable (abstractos, sin tabla)
usuarios	Usuarios, roles, permisos, municipios y organizaciones	Usuario, Municipio, Organizacion
puntos	Puntos de recepción, materiales y contenido educativo	Material, PuntoRecepcion, PuntoMaterial, ContenidoEducativo
campanias	Campañas, inscripciones y recolección	Campania, Participacion, RegistroRecoleccion
reportes	Reportes ambientales, seguimiento, apoyos y comentarios	ReporteAmbiental, SeguimientoReporte, Apoyo, Comentario

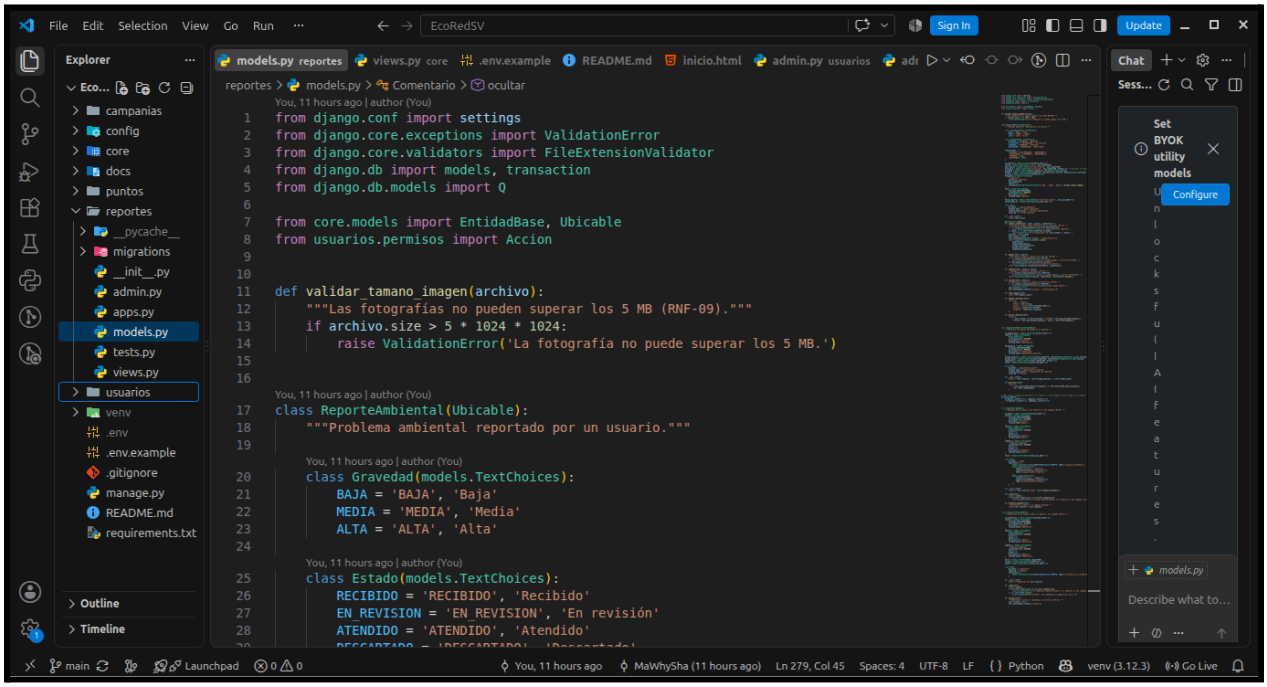
```
EcoRedSV/
├── config/             settings.py (AUTH_USER_MODEL, MEDIA_ROOT)
├── core/               models.py (clases abstractas), views.py, templates/
├── usuarios/           models.py, admin.py, permisos.py, migrations/
├── puntos/             models.py, admin.py, migrations/
├── campanias/          models.py, admin.py, migrations/
├── reportes/           models.py, admin.py, migrations/
├── docs/
├── manage.py
└── requirements.txt
```

Los módulos de la Semana 4 que trabajaban en memoria (core/base.py, core/datos_ejemplo.py y los archivos dominio.py) se eliminaron: su lógica pasó a ser los métodos de los modelos. Las versiones anteriores se conservan en el historial de Git.

Evidencia 1. Estructura del proyecto en VS Code

Descripción: aplicaciones desplegadas con sus archivos models.py y admin.py, y las carpetas migrations generadas.

[CAPTURA 01: Estructura del proyecto en VS Code]



4. Modelos y relaciones

Se implementaron 14 modelos concretos que generan 15 tablas: la tabla campania_material la crea Django a partir de una relación ManyToManyField. Todos heredan de models.Model a través de EntidadBase.

4.1 Modelos, tablas y relaciones

Modelo	Tabla	Relaciones
Municipio	municipio	Referenciado por usuarios, puntos, campañas y reportes.
Organizacion	organizacion	Referenciada por usuarios y campañas.
Usuario	usuario	Hereda de AbstractUser. ForeignKey opcional a Municipio y a Organizacion.

Modelo	Tabla	Relaciones
Material	material	ManyToMany con PuntoRecepcion y con Campania.
PuntoRecepcion	punto_recepcion	Hereda de Ubicable. ForeignKey a Municipio y al gestor (Usuario); ManyToMany con Material a través de PuntoMaterial.
PuntoMaterial	punto_material	Tabla intermedia con datos propios: ForeignKey a PuntoRecepcion y a Material, únicos juntos.
ContenidoEducativo	contenido_educativo	ForeignKey opcional a Material y obligatoria al autor (Usuario).
Campania	campania	Hereda de Ubicable. ForeignKey a Organizacion, PuntoRecepcion y creador (Usuario); ManyToMany con Material.
(automática)	campania_material	Generada por el ManyToManyField materiales de Campania.
Participacion	participacion	ForeignKey a Usuario y a Campania, únicos juntos.
RegistroRecoleccion	registro_recoleccion	ForeignKey a Campania, Material y al usuario que registró. CHECK: peso mayor que cero.
ReporteAmbiental	reporte_ambiental	Hereda de Ubicable. ForeignKey a Municipio y al autor (Usuario).
SeguimientoReporte	seguimiento_reporte	ForeignKey a ReporteAmbiental y al responsable (Usuario).
Apoyo	apoyo	ForeignKey a Usuario y a un reporte o una campaña. CHECK de un solo elemento y unicidad por usuario.
Comentario	comentario	ForeignKey a Usuario y a un reporte o una campaña. CHECK de un solo elemento.

Tipos de relación utilizados: ForeignKey para las relaciones de uno a muchos y ManyToManyField para las de muchos a muchos, en dos variantes: con tabla intermedia propia (PuntoMaterial, que guarda observaciones) y automática (campania_material). No se usó OneToOneField porque el modelo de datos de la Semana 3 no tiene relaciones uno a uno.

Nombres coherentes con la Semana 3: cada modelo define su tabla con `db_table`, su clave primaria como `id_<tabla>` y el nombre de cada clave foránea con `db_column`, para que las tablas de SQL Server coincidan con el diccionario de datos.

4.2 Código de los modelos

Se presentan los fragmentos más representativos; el código completo está en el repositorio.

core/models.py: clase abstracta Ubicable

```
class Ubicable(EntidadBase):
    direccion = models.CharField('dirección', max_length=250)
    latitud = models.DecimalField(max_digits=9, decimal_places=6)
    longitud = models.DecimalField(max_digits=9, decimal_places=6)
    municipio = models.ForeignKey(
        'usuarios.Municipio', on_delete=models.PROTECT, db_column='id_municipio',
    )

    class Meta:
        abstract = True  # no genera tabla propia

    def clean(self):
        super().clean()
        if not (self.LATITUD_MIN <= self.latitud <= self.LATITUD_MAX):
            raise ValidationError({'latitud': 'La latitud está fuera de El Salvador.'})

    def obtener_marcador(self):
        raise NotImplementedError('Cada subclase define su marcador.')
```

usuarios/models.py: modelo Usuario

```
class Usuario(AbstractUser, EntidadBase):
    class Rol(models.TextChoices):
        COMUNIDAD = 'COMUNIDAD', 'Comunidad'
        GESTOR = 'GESTOR', 'Gestor de residuos'
        ORGANIZACION = 'ORGANIZACION', 'Organización'
        ADMIN = 'ADMIN', 'Administrador'

    id_usuario = models.AutoField(primary_key=True)
    username = models.CharField(max_length=50, unique=True, db_column='nombre_usuario')
    email = models.EmailField(unique=True, db_column='correo')
    password = models.CharField(max_length=128, db_column='contrasena')
    nombre = models.CharField(max_length=100)
    correo_verificado = models.BooleanField(default=False)
    rol = models.CharField(max_length=20, choices=Rol.choices, default=Rol.COMUNIDAD)
    municipio = models.ForeignKey(Municipio, on_delete=models.PROTECT, null=True,
                                  blank=True, db_column='id_municipio')

    class Meta:
        db_table = 'usuario'

    def tiene_permiso(self, accion):
        if self.is_superuser:
            return True
        if accion in REQUIEREN_CORREO_VERIFICADO and not self.correo_verificado:
            return False
        return accion in PERMISOS_POR_ROL[RolPermiso(self.rol)]
```

puntos/models.py: relación de muchos a muchos con tabla intermedia

```
class PuntoRecepcion(Ubicable):
    gestor = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.SET_NULL,
                               null=True, blank=True, db_column='id_gestor')
    materiales = models.ManyToManyField(Material, through='PuntoMaterial',
                                        related_name='puntos', blank=True)

class PuntoMaterial(EntidadBase):
    punto = models.ForeignKey(PuntoRecepcion, on_delete=models.CASCADE, db_column='id_punto')
    material = models.ForeignKey(Material, on_delete=models.PROTECT, db_column='id_material')
    observaciones = models.CharField(max_length=200, blank=True)

    class Meta:
        db_table = 'punto_material'
        constraints = [
            models.UniqueConstraint(fields=['punto', 'material'], name='uq_punto_material'),
        ]
```

campanias/models.py: relaciones y comportamiento de Campania

```
class Campania(Ubicable):
    organizacion = models.ForeignKey('usuarios.Organizacion', on_delete=models.PROTECT,
                                     null=True, blank=True, db_column='id_organizacion')
    punto = models.ForeignKey('puntos.PuntoRecepcion', on_delete=models.SET_NULL,
                              null=True, blank=True, db_column='id_punto')
    creador = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.PROTECT,
                                db_column='id_creador')
    materiales = models.ManyToManyField('puntos.Material', blank=True,
                                        db_table='campania_material')

    def cambiar_estado(self, nuevo):
        if nuevo not in self.TRANSICIONES[self.estado]:
            raise ValueError(f'No se puede pasar de {self.estado} a {nuevo}.')
        self.estado = nuevo
        self.save(update_fields=['estado', 'actualizado_en'])

    def total_recolectado(self):
        filas = self.registros.values('material__nombre').annotate(total=Sum('peso_kg'))
        return {fila['material__nombre']: fila['total'] for fila in filas}
```

reportes/models.py: restricciones de Apoyo

```
UN_SOLO_ELEMENTO = (
    Q(reporte__isnull=False, campania__isnull=True)
    | Q(reporte__isnull=True, campania__isnull=False)
)

class Apoyo(EntidadBase):
    class Meta:
        db_table = 'apoyo'
        constraints = [
            models.CheckConstraint(condition=UN_SOLO_ELEMENTO, name='ck_apoyo_un_elemento'),
            models.UniqueConstraint(fields=['usuario', 'reporte'],
                                    condition=Q(reporte__isnull=False),
                                    name='uq_apoyo_usuario_reporte'),
        ]
```

4.3 Método __str__() y métodos propios

Cada modelo define cómo se muestra como texto y al menos un método relacionado con su comportamiento:

Modelo	__str__() devuelve	Métodos propios
Municipio	Nombre y departamento	listar_puntos()
Organizacion	Nombre	listar_campanias()
Usuario	Nombre y rol	tiene_permiso(), puede_editar(), verificar_correo()
Material	Nombre	puntos_que_lo_reciben(), mostrar_detalle()
PuntoRecepcion	Nombre	agregar_material(), quitar_material(), confirmar_informacion(), obtener_marcador()
PuntoMaterial	Punto y material	describir()
ContenidoEducativo	Título	publicar()
Campania	Título	cambiar_estado(), inscribir(), registrar_peso(), total_recolectado()

Modelo	__str__() devuelve	Métodos propios
Participacion	Usuario y campaña	confirmar_asistencia(), cancelar()
RegistroRecoleccion	Material y peso	describir()
ReporteAmbiental	Título	cambiar_estado(), apoyar(), comentar(), ocultar()
SeguimientoReporte	Reporte y cambio de estado	describir()
Apoyo	Usuario y elemento apoyado	elemento_apoyado()
Comentario	Autor	ocultar()

5. Aplicación de Programación Orientada a Objetos

Concepto	Cómo se aplica en los modelos
Herencia	Todos los modelos heredan de EntidadBase; PuntoRecepcion, Campania y ReporteAmbiental heredan además de Ubicable; Usuario hereda de AbstractUser, por lo que conserva el sistema de autenticación de Django.
Abstracción	EntidadBase y Ubicable son modelos abstractos (abstract = True): no generan tabla y sus campos se copian en las tablas de sus subclases.
Polimorfismo	obtener_marcador() se define distinto en cada modelo ubicable. La página de inicio recorre puntos, campañas y reportes y llama al mismo método sin conocer el tipo de cada objeto.
Encapsulamiento	El estado de campañas y reportes es de solo lectura en el panel: solo cambia mediante cambiar_estado(), que valida la transición; en los reportes, además, registra el cambio en el historial dentro de una transacción.
Validación	clean() comprueba las coordenadas, las fechas de la campaña, el estado y los materiales antes de registrar un peso; guardar() ejecuta estas validaciones antes de guardar.
Composición	Participacion y RegistroRecoleccion dependen de su campaña, y SeguimientoReporte de su reporte: se eliminan en cascada con ellos.
Reutilización	Usuario.tiene_permiso() usa las tablas de permisos del módulo permisos.py de la Semana 4, y los demás modelos lo consultan antes de inscribir, registrar pesos, apoyar, comentar o moderar.

Restricción de Django: los métodos abstractos de la Semana 4 usaban ABC. Los modelos de Django tienen su propia metaclase, incompatible con ABC, por lo que los métodos abstractos ahora lanzan NotImplementedError hasta que una subclase los define.

6. Ejecución de las migraciones

6.1 Preparación de la base de datos

En la Semana 4 las primeras migraciones habían creado la tabla de usuarios por defecto de Django (auth_user). Como el sistema usa un modelo Usuario propio, Django exige definirlo antes de la primera migración, así que la base ecoredsv se reinició. No se perdió información, porque no contenía datos reales.

```
USE master;
ALTER DATABASE ecoredsv SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
DROP DATABASE ecoredsv;
CREATE DATABASE ecoredsv;

USE ecoredsv;
CREATE USER ecoredsv_app FOR LOGIN ecoredsv_app;
ALTER ROLE db_owner ADD MEMBER ecoredsv_app;
```

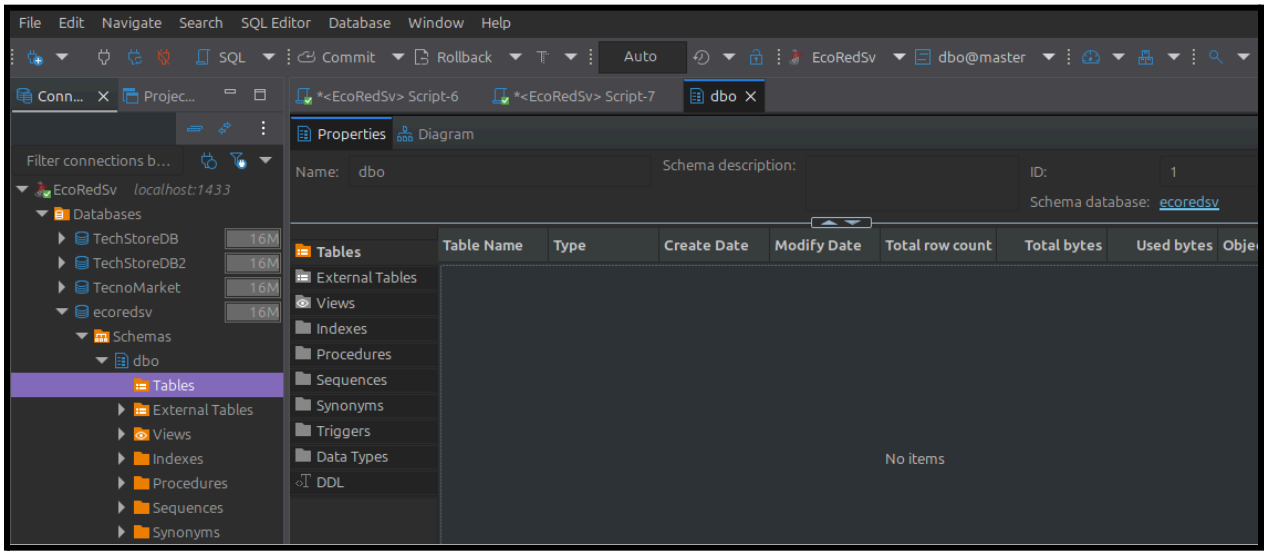
Luego se indicó a Django el modelo de usuario del proyecto y la carpeta de las fotografías de los reportes, en config/settings.py:

```
AUTH_USER_MODEL = 'usuarios.Usuario'
MEDIA_URL = 'media/'
MEDIA_ROOT = BASE_DIR / 'media'
```

Evidencia 2. Base de datos vacía en SQL Server

Descripción: la base ecoredsv recién creada, sin tablas, lista para las migraciones.

[CAPTURA 02: Base de datos vacía en SQL Server]



6.2 Creación y aplicación de las migraciones

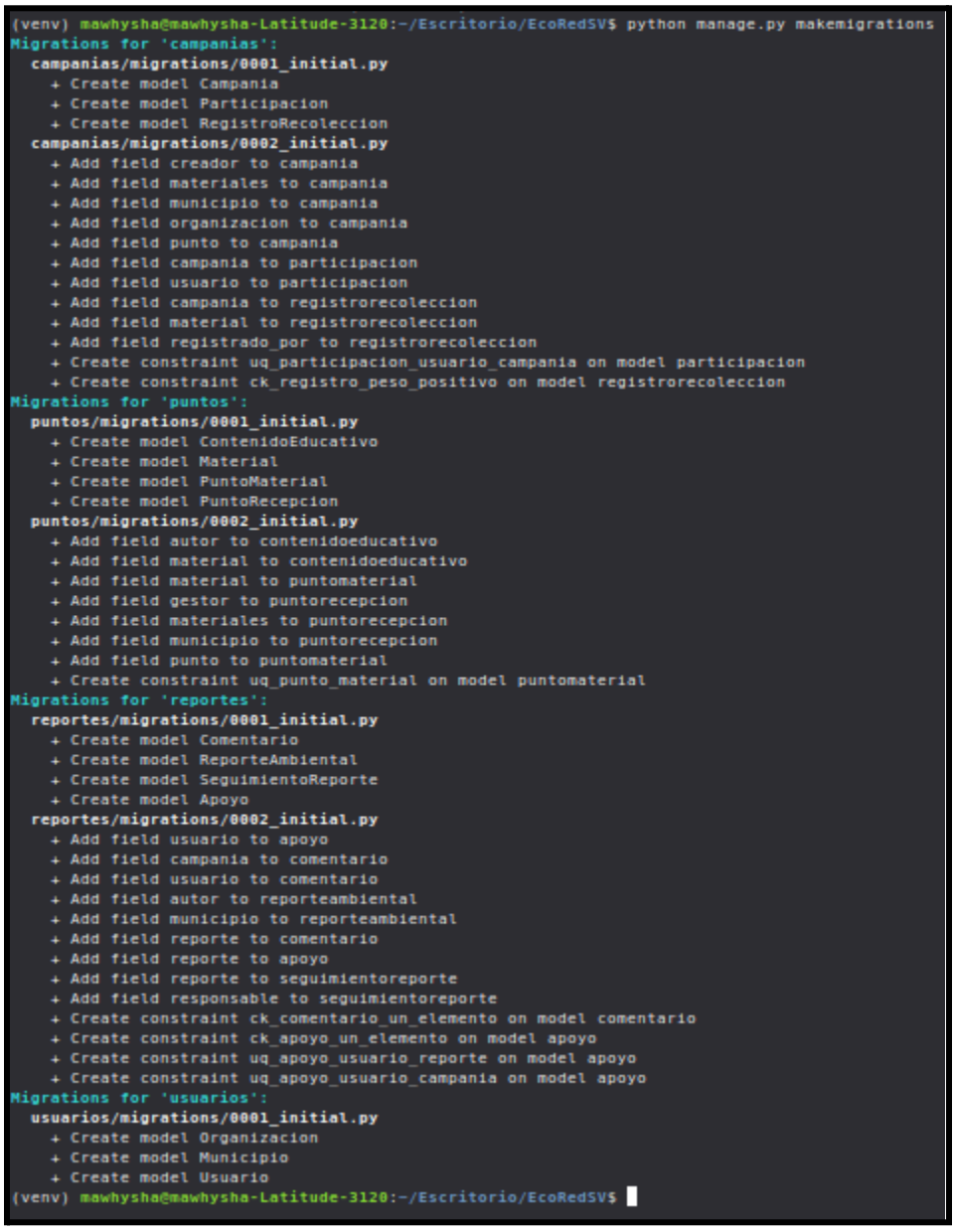
```
python manage.py check
python manage.py makemigrations
python manage.py migrate
```

makemigrations generó las migraciones de las cuatro aplicaciones con modelos. Las aplicaciones puntos, campanias y reportes tienen dos archivos (0001 y 0002) porque dependen entre sí: Django crea primero las tablas y luego agrega las relaciones entre ellas.

Evidencia 3. Creación de las migraciones

Descripción: salida de makemigrations con los modelos, campos y restricciones de cada aplicación.

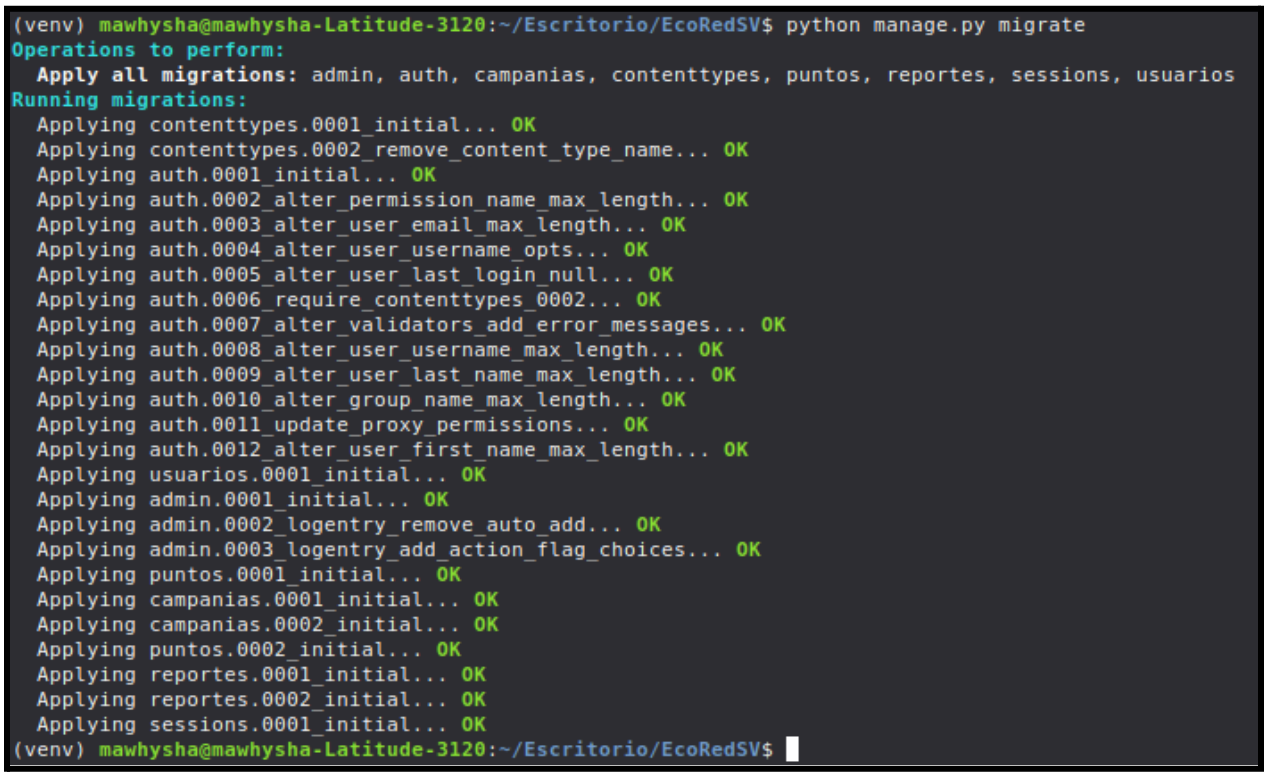
[CAPTURA 03: Creación de las migraciones]



Evidencia 4. Aplicación de las migraciones

Descripción: salida de migrate con todas las migraciones aplicadas con OK.

[CAPTURA 04: Aplicación de las migraciones]



6.3 Tablas creadas en SQL Server

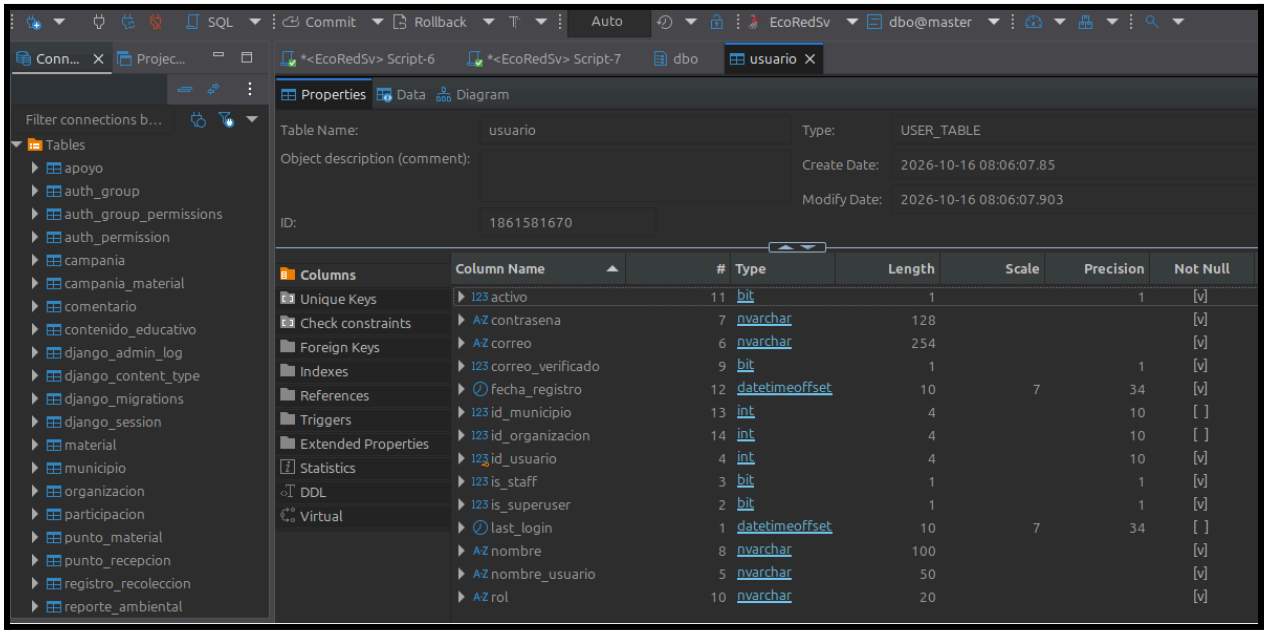
Se crearon las 15 tablas del diccionario de datos, junto con las tablas internas de Django para autenticación, permisos y sesiones. La tabla auth_user ya no existe: los usuarios se guardan en la tabla usuario.

Origen	Tablas
Diccionario de datos (15)	apoyo, campania, campania_material, comentario, contenido_educativo, material, municipio, organizacion, participacion, punto_material, punto_recepcion, registro_recoleccion, reporte_ambiental, seguimiento_reporte, usuario
Internas de Django	auth_group, auth_group_permissions, auth_permission, django_admin_log, django_content_type, django_migrations, django_session, usuario_groups, usuario_user_permissions

Evidencia 5. Tablas creadas en SQL Server

Descripción: lista de tablas en DBeaver y columnas de la tabla usuario, con los nombres del diccionario (id_usuario, nombre_usuario, correo, contrasena, rol, id_municipio).

[CAPTURA 05: Tablas creadas en SQL Server]



7. Panel de administración y datos de prueba

Los modelos se registraron en el panel de Django desde el admin.py de cada aplicación. El panel se adaptó al sistema:

- **Usuarios:** UsuarioAdmin propio, porque el modelo reemplaza los campos first_name y last_name por un solo campo nombre.
- **Puntos de recepción:** los materiales se agregan dentro del formulario del punto, con sus observaciones, y una acción confirma la información usando confirmar_informacion().
- **Campañas y reportes:** el estado es de solo lectura y se cambia con acciones que llaman a cambiar_estado(). Si la transición no está permitida, el panel muestra el error.
- **Contenido educativo y comentarios:** acciones para publicar y para ocultar (moderación) mediante los métodos de los modelos.

reportes/admin.py: el estado solo cambia mediante el método del modelo

```
@admin.register(ReporteAmbiental)
class ReporteAmbientalAdmin(admin.ModelAdmin):
    readonly_fields = ('estado',)
    inlines = [SeguimientoInline]
    actions = ['poner_en_revision', 'marcar_atendido', 'descartar', 'ocultar']

    def _cambiar_estado(self, request, queryset, nuevo):
        for reporte in queryset:
            try:
                reporte.cambiar_estado(nuevo, request.user, 'Cambio desde el panel.')
            except (ValueError, PermissionError) as error:
                self.message_user(request, f'{reporte}: {error}', messages.ERROR)
```

El superusuario se creó con `python manage.py createsuperuser`. Su manejador personalizado (UsuarioManager) le asigna el rol de administrador y el correo verificado automáticamente.

7.1 Datos de prueba ingresados

Todos los datos son ficticios y se ingresaron desde el panel de administración:

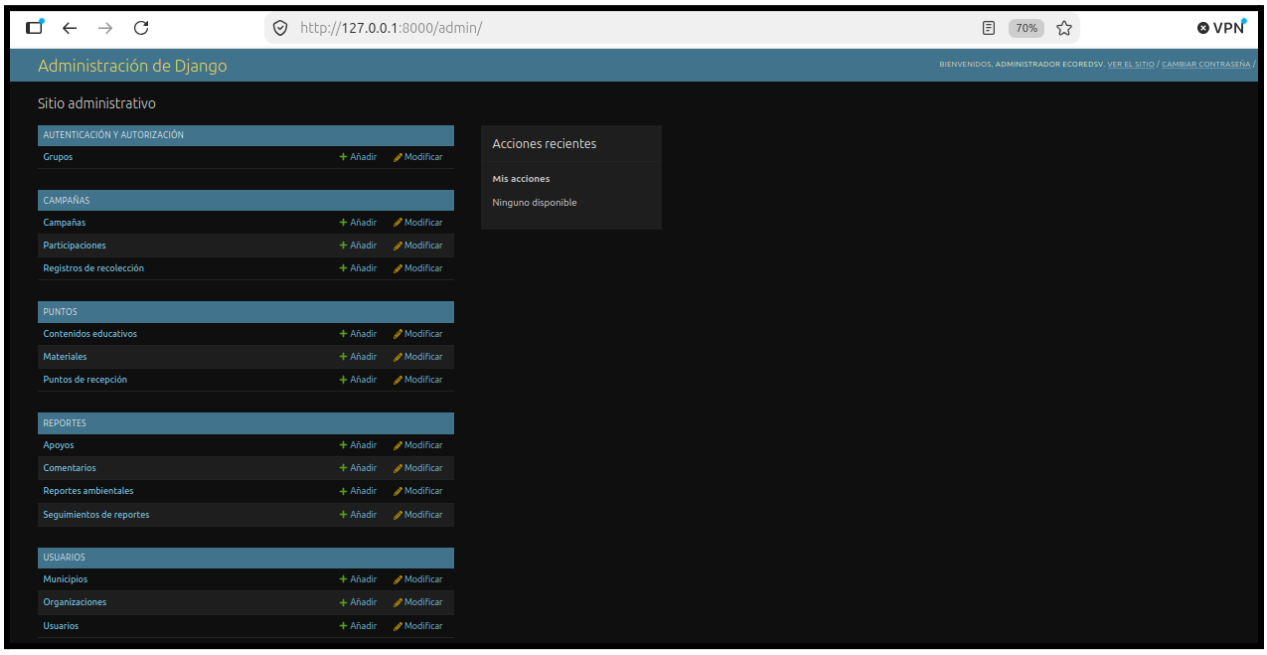
Modelo	Registros	Detalle
Usuarios	4	admin y un usuario de cada rol: comunidad, gestor y organización, con correo verificado.
Municipios	2	Chalatenango y Tejutla.
Organizaciones	1	Organización de prueba (ONG).
Materiales	4	Botellas PET, latas de aluminio, cartón y botellas de vidrio.
Puntos de recepción	2	Uno con información confirmada y otro por confirmar, con sus materiales.
Campañas	1	Jornada en curso con tres materiales, una inscripción y tres registros de peso.
Reportes ambientales	2	Gravedad alta y media; uno puesto en revisión con su historial.
Apoyos y comentarios	5	Apoyos a reportes y a la campaña, y un comentario.
Contenido educativo	1	Publicado mediante la acción del panel.

7.2 Evidencias

Evidencia 6. Panel de administración

Descripción: página principal del panel con los modelos registrados de las aplicaciones usuarios, puntos, campañas y reportes.

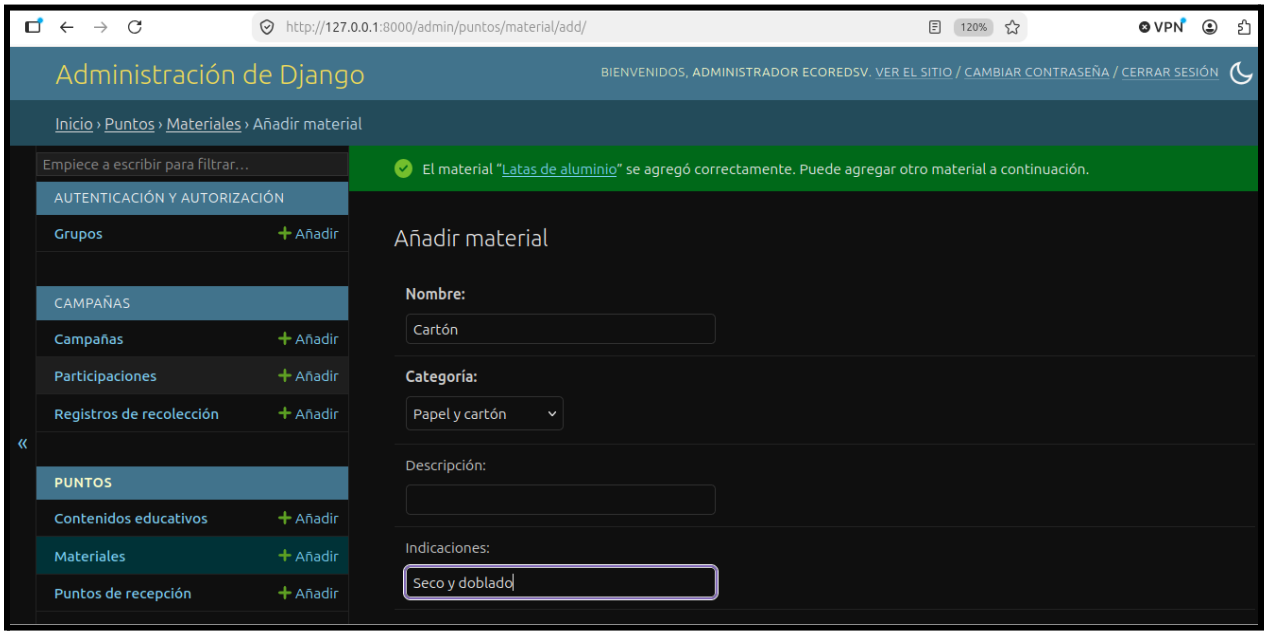
[CAPTURA 06: Panel de administración]



Evidencia 7. Registro de un material

Descripción: formulario del panel para agregar un material, con su categoría e indicaciones.

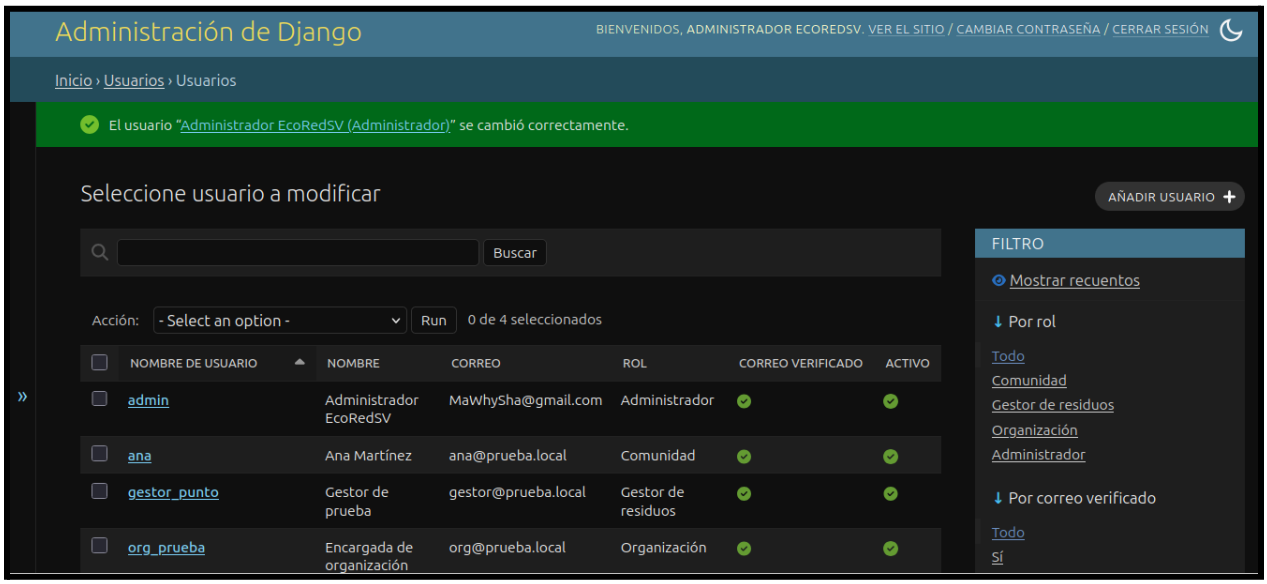
[CAPTURA 07: Registro de un material]



Evidencia 8. Usuarios registrados

Descripción: lista de usuarios en el panel con su rol y su correo verificado.

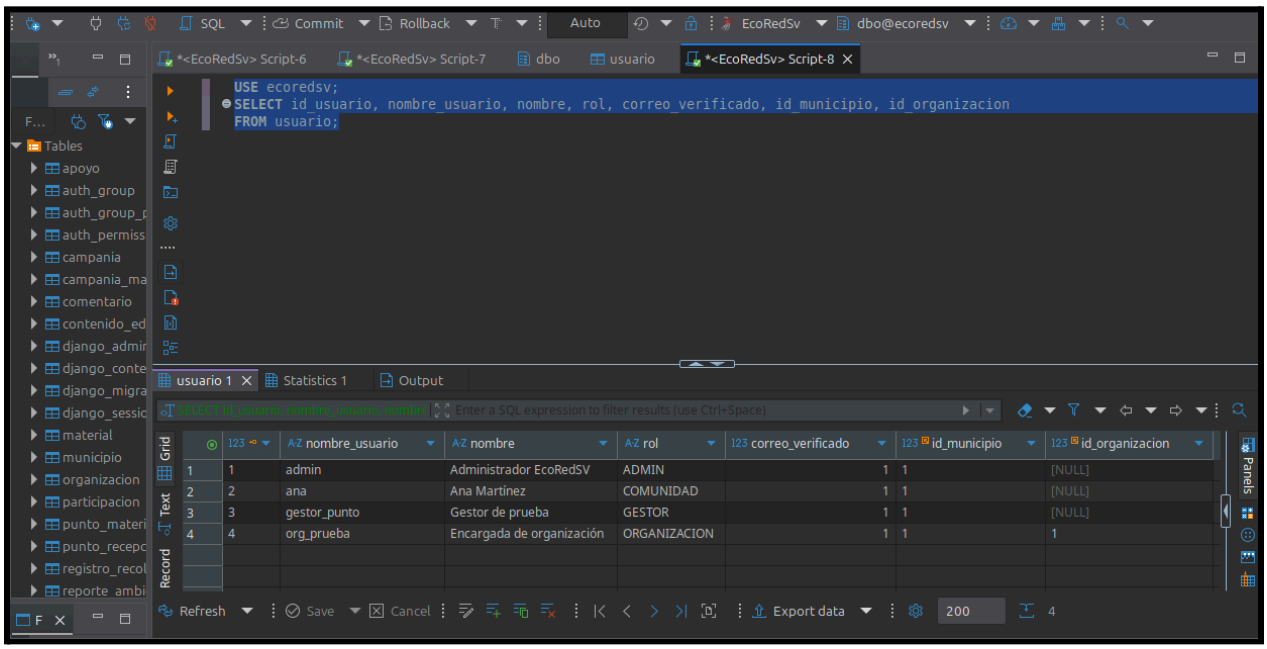
[CAPTURA 08: Usuarios registrados]



Evidencia 9. Datos reflejados en SQL Server

Descripción: consulta a la tabla usuario en DBeaver: los usuarios ingresados en el panel están guardados en la base de datos, con su rol, municipio y organización.

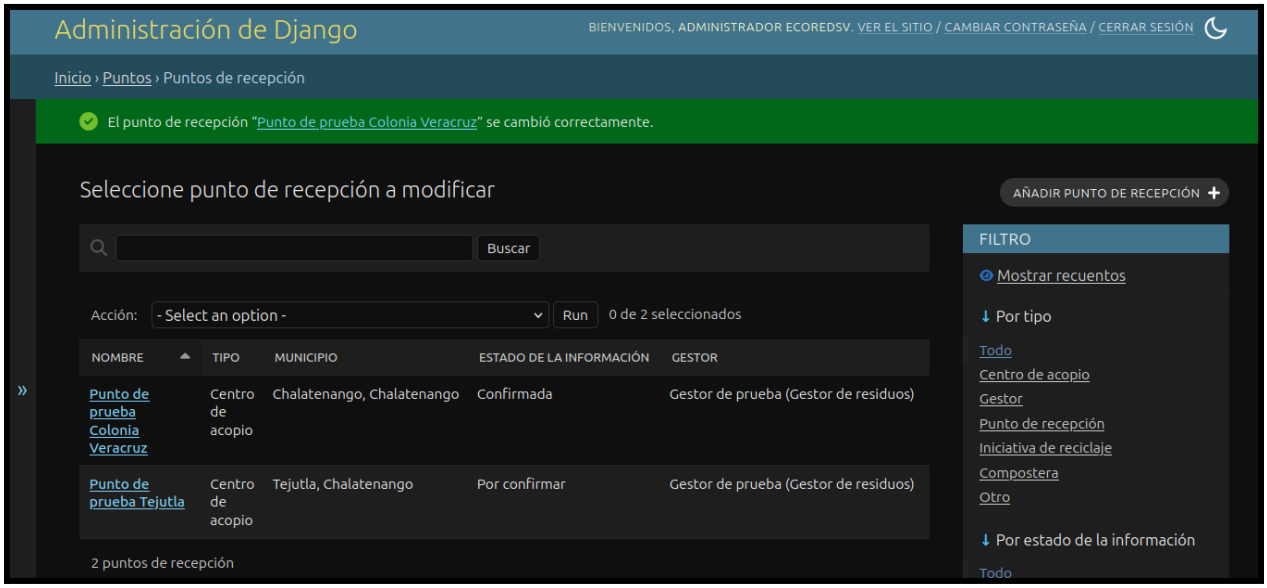
[CAPTURA 09: Datos reflejados en SQL Server]



Evidencia 10. Puntos de recepción

Descripción: lista de puntos: uno con la información confirmada mediante la acción del panel y otro por confirmar.

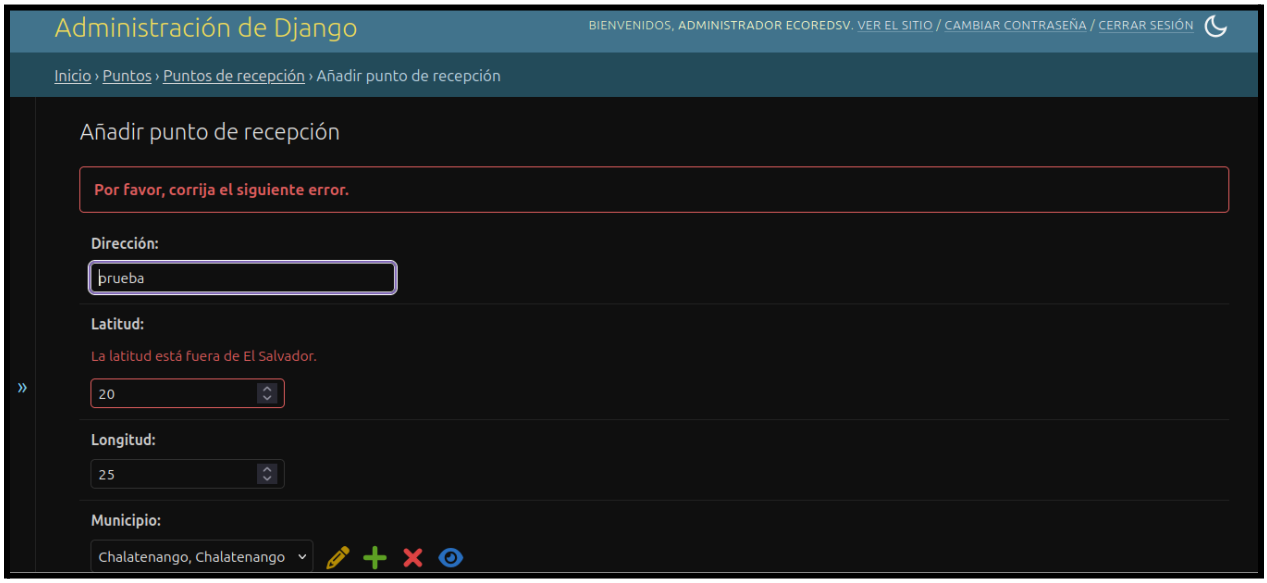
[CAPTURA 10: Puntos de recepción]



Evidencia 11. Validación de un punto de recepción

Descripción: el panel rechaza una coordenada fuera de El Salvador con el mensaje definido en clean() del modelo Ubicable.

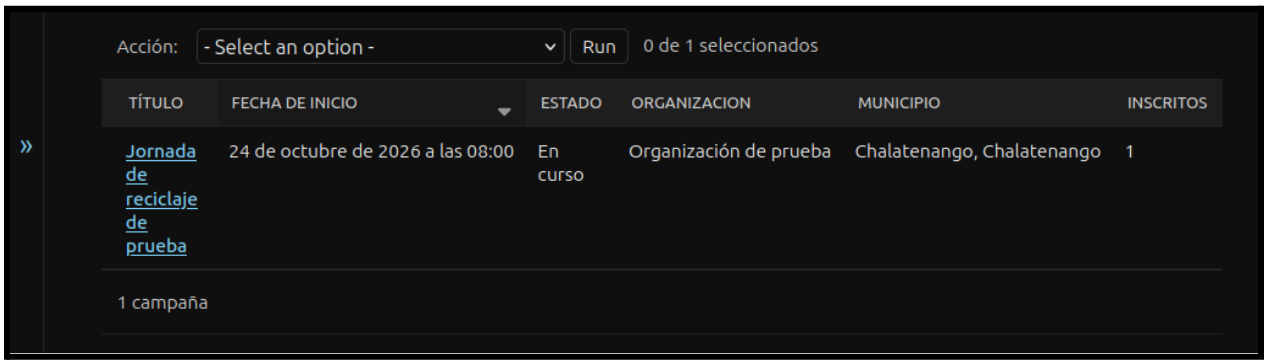
[CAPTURA 11: Validación de un punto de recepción]



Evidencia 12. Campaña en curso

Descripción: jornada de reciclaje iniciada con la acción del panel, que usa cambiar_estado().

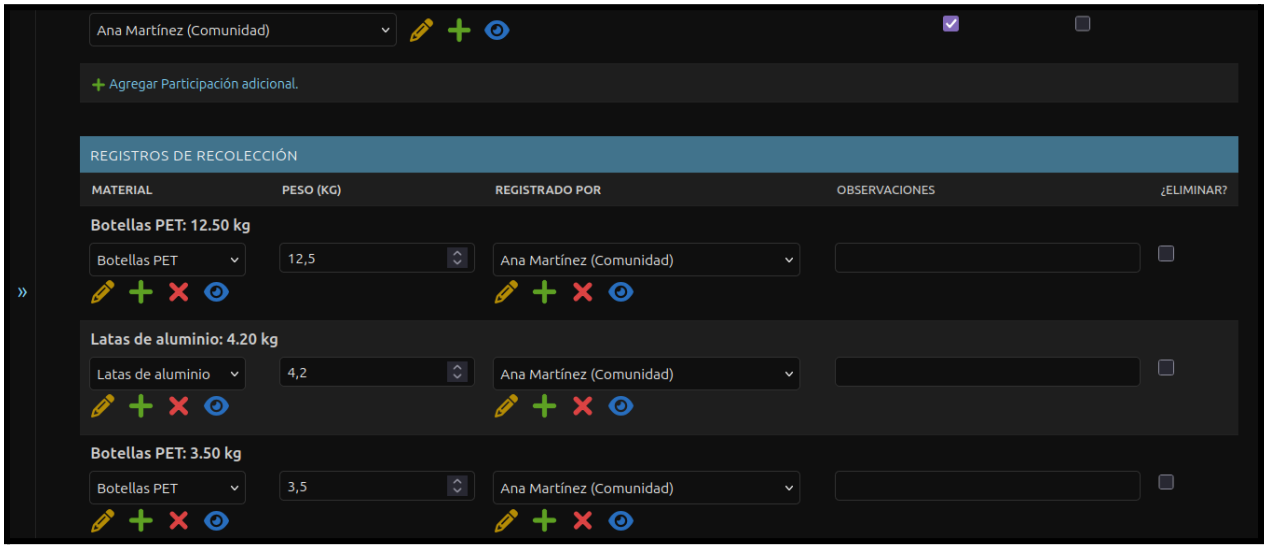
[CAPTURA 12: Campaña en curso]



Evidencia 13. Registro de materiales recolectados

Descripción: inscripción de un usuario y registros de peso por material dentro de la campaña.

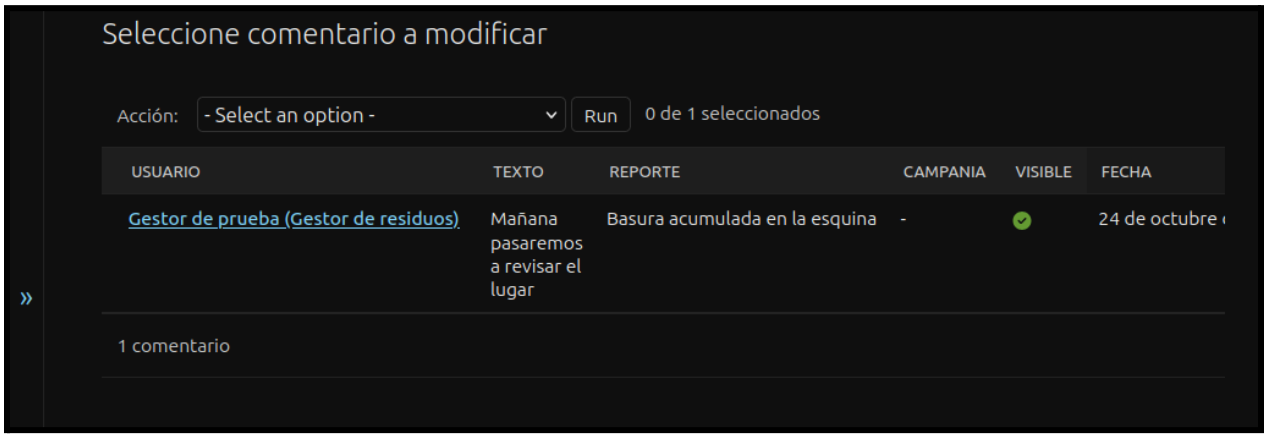
[CAPTURA 13: Registro de materiales recolectados]



Evidencia 14. Comentario registrado

Descripción: comentario de un usuario sobre un reporte ambiental.

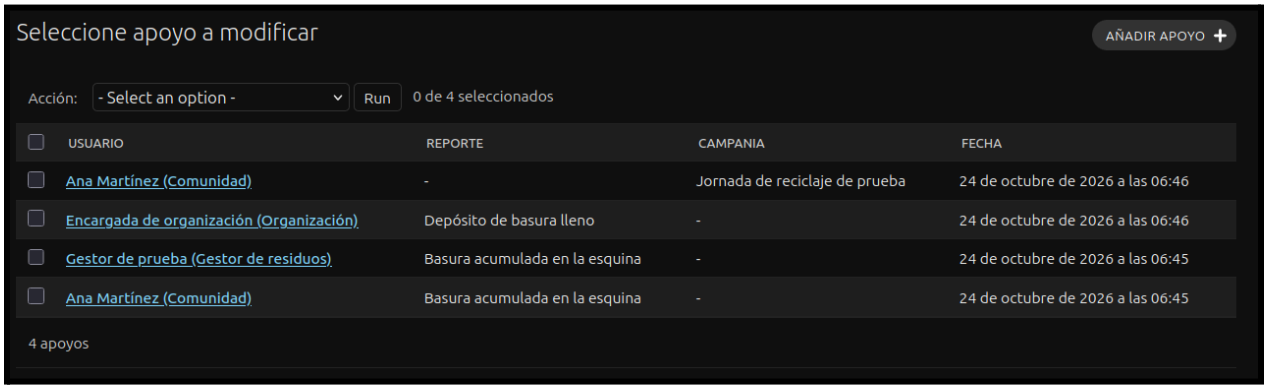
[CAPTURA 14: Comentario registrado]



Evidencia 15. Apoyos registrados

Descripción: apoyos de los usuarios a reportes y a la campaña.

[CAPTURA 15: Apoyos registrados]



Evidencia 16. Página de inicio con datos reales

Descripción: conexión con SQL Server y cantidad de registros de cada tabla principal.

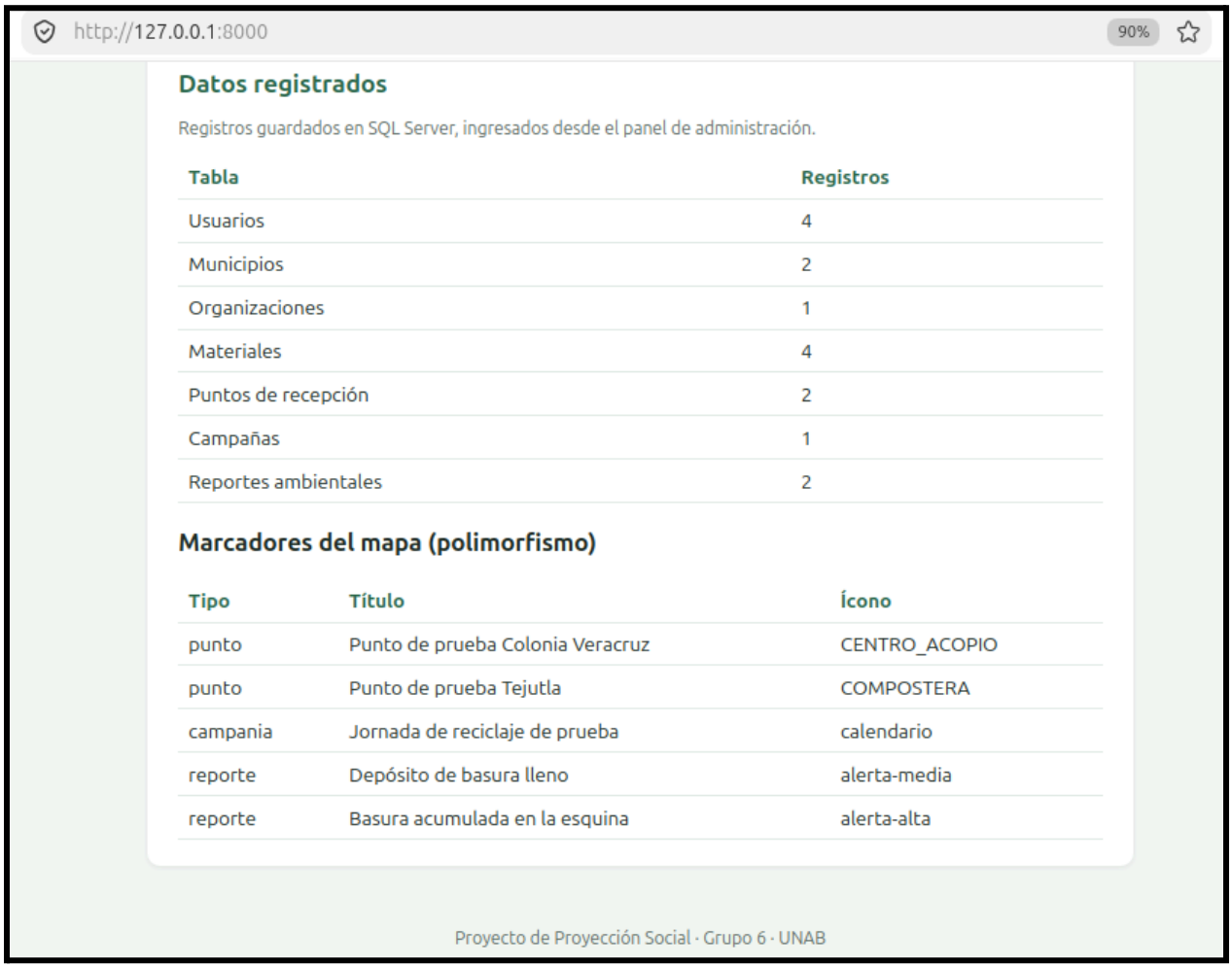
[CAPTURA 16: Página de inicio con datos reales]



Evidencia 17. Marcadores del mapa

Descripción: puntos, campaña y reportes generando su marcador con obtener_marcador(), cada uno a su manera (polimorfismo).

[CAPTURA 17: Marcadores del mapa]



8. Historial de commits

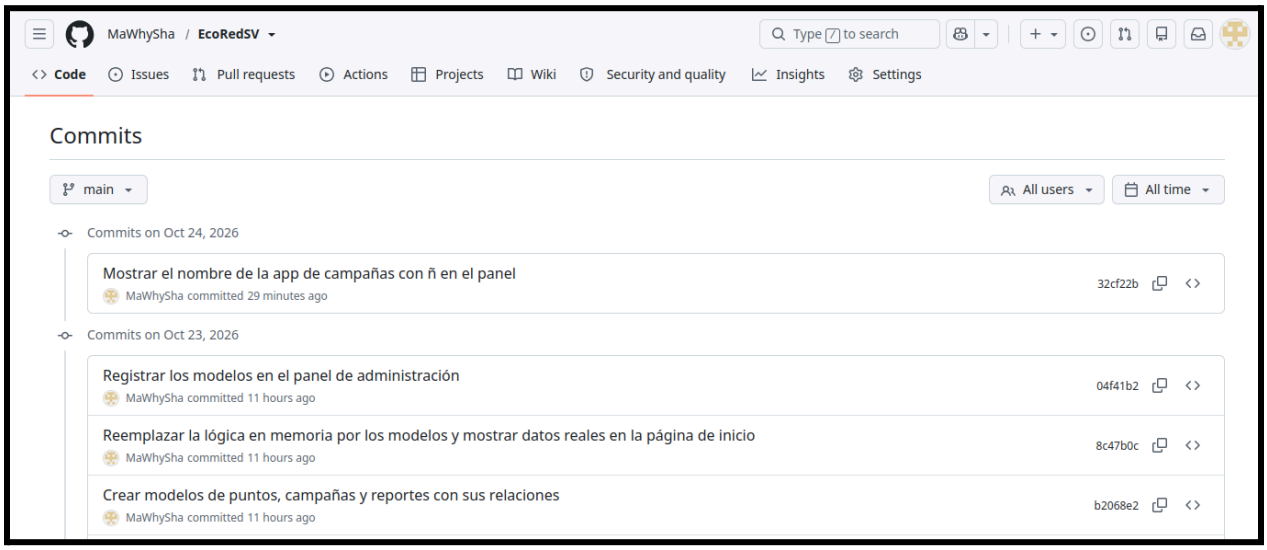
El avance de la semana se subió al repositorio en commits separados por tema.

Commit	Descripción
32cf22b	Mostrar el nombre de la app de campañas con ñ en el panel
04f41b2	Registrar los modelos en el panel de administración
8c47b0c	Reemplazar la lógica en memoria por los modelos y mostrar datos reales en la página de inicio
b2068e2	Crear modelos de puntos, campañas y reportes con sus relaciones
c71ee2f	Crear modelos base y de usuarios con sus migraciones
df30648	Configurar modelo de usuario propio y carpeta de archivos subidos

Evidencia 18. Historial de commits en GitHub

Descripción: commits de la Semana 5 en el repositorio.

[CAPTURA 18: Historial de commits en GitHub]



9. Dificultades encontradas y soluciones

N.º	Dificultad	Causa	Solución
1	La base de datos tenía la tabla auth_user de Django y no admitía el modelo Usuario propio.	Django exige definir el modelo de usuario personalizado antes de la primera migración, y en la Semana 4 ya se había migrado con el modelo por defecto.	Se reinició la base ecoredsv, se volvió a dar acceso al usuario ecoredsv_app y se configuró AUTH_USER_MODEL antes de migrar.

N.º	Dificultad	Causa	Solución
2	Las clases abstractas con ABC no funcionaban como modelos.	Los modelos de Django usan su propia metaclase, incompatible con la de ABC.	Se usaron modelos abstractos (abstract = True) y los métodos abstractos lanzan NotImplementedError.
3	Django crea por defecto nombres como usuarios_usuario y columnas como municipio_id, distintos al diccionario de datos.	Django genera los nombres a partir de la aplicación y el campo.	Se definieron db_table, claves primarias id_<tabla> y db_column en cada modelo.
4	El panel de usuarios por defecto de Django no funcionaba con el modelo propio.	UserAdmin espera los campos first_name y last_name, que el modelo reemplaza por nombre.	Se creó UsuarioAdmin con sus propios grupos de campos.
5	Clases repetidas: Campania, PuntoRecepcion y ReporteAmbiental existían en los archivos dominio.py y en models.py.	La lógica de la Semana 4 trabajaba en memoria y ahora vive en los modelos.	Se eliminaron los archivos dominio.py, base.py y datos_ejemplo.py; la página de inicio ahora consulta SQL Server.
6	El panel rechazó la contraseña de los usuarios de prueba.	El validador de Django no acepta contraseñas parecidas al correo o al nombre del usuario.	Se eligió una contraseña sin relación con los datos del usuario.
7	El servidor no iniciaba: "That port is already in use".	Había otra instancia del servidor ocupando el puerto 8000.	Se detuvo con fuser -k 8000/tcp y se volvió a iniciar.

10. Enlaces

Recurso	Enlace	Permisos
Google Docs principal	https://docs.google.com/document/d/1aSZ90jYoQQPBdBpHOCxw4lpaN_MYYtPjztaSrMe9FOA/edit?usp=drivesdk	Visualización habilitada para el docente
Repositorio	https://github.com/MaWhySha/EcoRedSV.git	Visualización habilitada para quien tenga el enlace